

Minimum Spanning Trees (MST)

A **Minimum Spanning Tree (MST)** is a subset of the edges of a connected, edge-weighted **undirected graph** that connects all the vertices together, without any cycles and with the minimum possible total edge weight.

- **Properties:**

- It must connect all V vertices.
- It must use exactly $V-1$ edges.
- It must not contain any **cycles**.

- **Application:** Network design (e.g., laying cables, optimizing communication links) where minimizing total cost is the goal.

Algorithm	Primary Approach	Data Structure	Time Complexity	Note
Prim's	Greedy (Vertex-focused)	Min Priority Queue	$O(E + V \log V)$	Grows the tree from a single starting vertex.
Kruskal's	Greedy (Edge-focused)	Disjoint Set Union (DSU)	$O(E \log E)$ or $O(E \log V)$	Processes edges in increasing order of weight.

Prim's Algorithm

Prim's Algorithm builds the MST one vertex at a time. It works by growing a single connected component (the MST) outwards from a starting vertex.

- **Principle:** At each step, it selects the **minimum weight edge** that connects a vertex **already in the MST** to a vertex **not yet in the MST**.
- **Data Structure:** A **Min Priority Queue** is used to store and quickly retrieve the minimum weight edge crossing the boundary between the growing MST and the rest of the graph.
- **Process:**
 1. Start at an arbitrary source vertex s .
 2. Initialize a Min Priority Queue with the edges connected to s .
 3. While the PQ is not empty and not all vertices are included:
 - Extract the edge (u, v) with the minimum weight from the PQ.
 - If v is already included, skip this edge (it creates a cycle in the partial MST).
 - Otherwise, add v to the MST and relax all its outgoing edges by adding them to the PQ.

Sumarised Prim's Algorithm

Prim's Algorithm basically builds the Minimum Spanning Tree one vertex at a time by repeatedly selecting the vertex outside the current tree that can be connected by the single

cheapest edge from any vertex already inside the tree.

C++ Code (Conceptual Core)

```
#include <queue>
#include <vector>
#include <utility>
#include <limits>

void prim_mst(int V, const std::vector<std::pair<int, int>> adj[], int
    // pair<weight, vertex>
    std::priority_queue<std::pair<int, int>,
                        std::vector<std::pair<int, int>>,
                        std::greater<std::pair<int, int>>> pq;

    std::vector<int> key(V, std::numeric_limits<int>::max());
    std::vector<bool> inMST(V, false);

    key[start] = 0;
    pq.push({0, start}); // {weight, vertex}
    int mst_weight = 0;

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        if (inMST[u] == true) continue;

        inMST[u] = true;
        mst_weight += key[u]; // Add the weight of the edge used to rea

        // Relax adjacent edges
        for (auto& edge : adj[u]) {
            int v = edge.first;
            int weight = edge.second;

            if (inMST[v] == false && weight < key[v]) {
                key[v] = weight;
                pq.push({key[v], v});
            }
        }
    }

    // std::cout << "Total MST Weight: " << mst_weight << std::endl;
}
```

Union-Find Disjoint Set

The Union-Find Disjoint Set (UFDS) data structure is characterized by three core operations that allow it to efficiently manage a collection of disjoint sets:

1. Make-Set (x)

- 2. **Find(x)**
 - 3. **Union(x, y)**
-

✂️ Core UFDS Operations

1. Make-Set (x) (or Initialize)

- **Purpose:** Creates a new set whose only member is the element x .
- **Action:** When a UFDS structure is initialized for a collection of n items, this operation is called n times. Each element is initially its own representative (or root) of a new, distinct set.
- **Time Complexity:** Typically $O(1)$.

2. Find(x)

- **Purpose:** Determines which set an element x belongs to.
- **Action:** Returns the **representative** (or root) of the set containing x . Two elements, x and y , are in the same set if and only if $\text{Find}(x)$ returns the same representative as $\text{Find}(y)$.
- **Optimization (Path Compression):** This is a key optimization where, during the Find operation, every node encountered along the path from x to the root is directly reattached to the root. This flattens the structure and significantly speeds up future operations.
- **Time Complexity:** Nearly constant amortized time, $O(\alpha(n))$, where α is the inverse Ackermann function.

3. Union(x, y)

- **Purpose:** Merges the set containing element x and the set containing element y into a single set.
 - **Action:**
 1. First, it checks if x and y are already in the same set using $\text{Find}(x)$ and $\text{Find}(y)$.
 2. If they belong to different sets, the two sets are merged by making the root of one set point to the root of the other.
 - **Optimization (Union by Rank/Size):** This strategy always attaches the smaller tree/set to the root of the larger tree/set. This prevents the tree structure from becoming overly deep and maintains the nearly constant time complexity for subsequent operations.
 - **Time Complexity:** Nearly constant amortized time, $O(\alpha(n))$.
-

Summary Table on UFDS

DSU (Disjoint Set Union) and UFDS (Union-Find Disjoint Set) refer to the same data structure.

Operation	Description	Purpose	Amortized Time Complexity
-----------	-------------	---------	---------------------------

Make-Set (x)	Create a new set with x as the only member.	Initialization of a new component.	$O(1)$
Find (x)	Return the representative of the set containing x .	Check set membership and connectivity.	$O(\alpha(n))$
Union (x, y)	Merge the set containing x and the set containing y .	Connect two previously disjoint components.	$O(\alpha(n))$

□ Kruskal's Algorithm

Kruskal's Algorithm is a greedy algorithm that sorts all graph edges by weight and then iteratively adds the next shortest edge to the Minimum Spanning Tree (MST), provided that the edge does not form a cycle with the edges already chosen, until all vertices are connected.

- **Principle:** Select edges in **non-decreasing order of weight** and add them to the MST if and only if they do not create a **cycle** with the edges already selected.
- **Data Structure:**
 1. A way to **sort all edges** by weight (e.g., using `std::sort` or a Min Priority Queue).
 2. A **Disjoint Set Union (DSU)** structure to efficiently detect cycles.
- **Cycle Detection (DSU):**
 - The `find` operation determines which set a vertex belongs to.
 - If the two vertices of an edge are already in the **same set**, adding the edge creates a cycle (they are already connected).
 - If they are in **different sets**, the edge is added, and their sets are **unioned** (they are now connected).

C++ Code (Conceptual Core)

```
#include <algorithm> // For std::sort
#include <vector>
#include <tuple>

// (Conceptual DSU function - assumes DSU structure is implemented)
// int find_set(int i);
// void union_sets(int i, int j);

// tuple<weight, u, v>
void kruskal_mst(int V, std::vector<std::tuple<int, int, int>>& edges)
    // 1. Sort all edges by weight
    std::sort(edges.begin(), edges.end());

    // (Initialize DSU structure for V vertices)

    int mst_weight = 0;
    int edges_count = 0;

    for (const auto& edge : edges) {
        int weight = std::get<0>(edge);
```

```
int u = std::get<1>(edge);
int v = std::get<2>(edge);

// 2. Check for cycle using DSU
// if (find_set(u) != find_set(v)) {
//     union_sets(u, v);
//     mst_weight += weight;
//     edges_count++;
// }

// Stop once V-1 edges are found
if (edges_count == V - 1) break;
}
// std::cout << "Total MST Weight: " << mst_weight << std::endl;
}
```